

LeetCode 2487

Remove Nodes From a Linked List

Adobe

Google

Problem Statement & Concept

Goal: Remove every node which has a node to its right having a value greater than its value.

Example:

Input: $5 \rightarrow 2 \rightarrow 13 \rightarrow 3 \rightarrow 8$

Output: $13 \rightarrow 8$

Hence, this question should immediately remind us of the *Next Greater Element* pattern.

Node Value	Greater on Right?	Action
5	Yes	Delete
2	Yes	Delete
13	No	Keep
8	No	Keep
3	Yes	Delete

Core Concept:

In a singly linked list, keeping track of a larger element on the right is hard, but on the left is easy due to `maxSeen`.

So, reverse the list!

Approach 1: Optimal (Reverse)

Strategy: Reverse → Process → Reverse → Return

Walkthrough:

Original: 5 → 2 → 13 → 3 → 8

Reversed: 8 → 3 → 13 → 2 → 5

1. max = 8 — Keep (Next)
2. 3 < 8 — Delete
3. 13 > 8 — Update max to 13
4. 2 < 13 — Delete
5. 5 < 13 — Delete

Complexity: Time Complexity = $O(n)$, Space Complexity = $O(1)$

```
head = reverseList(head); int max = head->val; ListNode * curr = head; // go till last node while ( curr && curr->next ) { if ( curr->next->val < max ) { // Reinitialize connection // delete that node } else { curr = curr->next; max = curr->val; } } return reverseList(head);
```

Approach 2: Monotonic Stack

Concept: Simply traverse the linked list. But whenever a big value arrives, empty the stack to ensure a monotonically decreasing stack.

Complexity: Time Complexity = $O(n)$, Space Complexity = $O(n)$

```
vector<ListNode*> st; ListNode* curr = head; while (curr) { while (!st.empty() && st.back()->val < curr->val) { st.pop_back(); } st.push_back(curr); curr = curr->next; } // Now we only have surviving nodes so start reconstruction for (int i = 0 ; i <
```

```
st.size() - 1 ; i++) { st[i]->next = st[i+1]; } st[st.size()-1]-  
>next = nullptr; return st[0];
```

© Copyright - Authored by Arpit Raj, LNMIIT Jaipur, June 25, 2026. All rights reserved.